V5 自然口播稿 精修版

对应 24 页主讲。正文和媒体预算 850 秒，另留约 50 秒交接，合计约 15 分钟。实际时长未真人计时。

本组实践暂时搁置。A/B/C/D 尚未对应成员姓名。页脚来源不用朗读；第 11 页短片边播放边解释，播放失败讲同页静态图。

A 问题与动机、案例进入

01 智能体在大模型产业研发中的应用 预算 15 秒

大家好。我们关注的是智能体在大模型产业研发中的应用。AI 已经开始参与研发 AI，那么，这些改进能不能一轮一轮继续下去？这是今天想回答的问题。

02 机器设计更好机器：一个早于大模型的问题 预算 30 秒

这个问题其实比大模型早得多。1965 年，统计学家 Good 就讨论过：如果机器能够参与设计更好的机器，改进会不会不断继续？当时主要是思想推演，今天则开始出现能检查的研发实例。我们不急着预测什么时候会发生智能爆炸，先把问题落到一项真实任务上：Agent 接过了什么工作，交付了什么，又怎样知道它做得更好？

03 研发 AI，需要反复做实验和评测 预算 35 秒

可以先想象研究员面前的一轮实验：有了一个想法，还得写代码、配置训练、读日志，最后判断结果值不值得保留。比如，模型变得更少迎合用户了，会不会同时变得不擅长数学？一次实验失败，下次又该从哪里接着试？这些工作有明确工具，也有需要判断的环节。课程里讲的工具调用、记忆和评测，进入大模型研发以后，就对应这些具体动作。

04 自我修改怎样才值得保留 预算 40 秒

自我修改听起来很诱人，但有一个老问题：凭什么保留这次修改？这里有两种思路。Gödel Machine 要求先证明修改有益，再允许修改；这个保证很强，但真实研发很难事先把所有后果都证明清楚。另一种思路是产生候选，实际运行，再用评测筛选，神经架构搜索和后面的代码自改研究更接近这条路线。它们体现的是验证方式的差异。今天的案例主要依赖实验，所以评测测了什么、漏了什么，会直接影响结论。

05 沿着研发链条找答案 预算 20 秒

下面先跟着一个改模型的任务走：任务怎么交给 Agent，方法怎么进入模型，结果又怎么用于下一次尝试。看完这些具体工作，再回到题目：这些改进能不能让下一轮研发做得更好?

06 为什么重点看 Anthropic 与 OpenAI 预算 25 秒

我们重点选这两家，是想展开研发中的两个环节。Anthropic 的公开研究能看到训练方法和目标模型的变化，OpenAI 作者的案例能看到评测任务怎样执行、怎样留下记录。Google 和开放研究再补上算法、策略和代码。一起看，就能知道 Agent 进入了哪些研发工作。

07 任务：让模型少迎合错误观点 预算 40 秒

先看 Anthropic Fellows 项目的一项公开对齐研究。任务是让模型少迎合错误观点：用户说错了，模型也应该能指出问题。请看图上三个角色。Claude Opus4.8 作为研究 Agent，想办法、写实现和调用训练工具；Qwen3.5-2B 是接受训练的目标模型；评测器检查改善和能力副作用。这里“Agent 研发 AI”的具体含义，就是它能通过工具改变另一个模型。下面看它怎样把一个想法变成可检查的实验。

B 模型改进机制与结果

08 一轮研究需要交付能运行的方法 预算 40 秒

从左往右看，一轮研究先读资料、提出假设，在看到本轮评测结果之前冻结小论文，再提交实现。这样就能对照原来的假设检查结果。基本规则不允许利用评测数据，也禁止从研究 Agent 或更强模型蒸馏。代码经过监控，训练工具更新目标模型，最后由外部评测打分。典型方法的预算约一张 H200、半小时。所以，Agent 必须把想法做成可运行的方法，同时遵守资源和数据边界。

09 下一次尝试，能读到之前的成功与失败 预算 40 秒

一次尝试结束以后，结果不会随着聊天结束就消失。多个研究 Agent 并行探索，方法、得分和研究记录进入共享资料。新的会话可以读取这些记录，知道哪些方向试过，再提出下一种方法。所以这里的记忆首先是一套外部工程机制：即使换了会话，研究也能接续。请注意这个回流方向，它把经验带回下一次尝试，却没有因此更新研究 Agent 的模型权重。下一页我们就看一个真正涉及目标模型变化的方法。

10 激活引导改变教师的回答倾向 预算 50 秒

这项研究在开放权重实验中找到的一条成功路线，是激活引导加自蒸馏。先只看第一步。模型回答问题时，中间会形成内部表示；方法对这些表示施加方向性的干预，让回答更倾向于指出错误，而不是顺着用户说。图上的教师，就是目标模型经过引导后的版本，并不是另外请来的一个更强模型。这个干预作用在模型内部，不是把一条新指令拼到用户提示里。可以把这一步理解为，先临时改变它的回答倾向。此时学生还没有经过训练：去掉这项推理时干预，刚才的变化能不能留下来？这就需要第二步。

11 蒸馏训练把行为变化写进学生权重 预算 50 秒

请跟着短片看三个动作。首先，引导后的教师给出输出概率分布；学生也给出自己的分布。然后，训练目标用 KL 散度衡量两者的差异，并据此调整学生参数。最后，学生不用刚才的临时引导，正常推理，再接受外部评测。把教师和学生分开讲，就是为了看清临时干预与参数训练发生在不同阶段。也就是说，第一步改变教师的输出，第二步才试着把这种行为写进学生权重。短片是教学示意，不是论文的实验录像。分布更接近，只说明训练目标被优化了；学生是否真的少迎合、是否损失能力，仍要看最后的评测。

12 改善空间：先弄清百分数的分母 预算 40 秒

在展示真实数字之前，先看这个教学例子。假设基线是 50 分，训练以后 70 分，最优是 100 分。实际多了 20 分，但原来有 50 分的改善空间，所以关闭了 40%的改善空间。分母是原来还能提高多少，不是测试题的总数，因此这个 40%不能读成通过率。如果换了基线，百分数也会随原有改善空间改变。论文还会把多个基准汇总，具体计算放在备份。大家只要先记住这条尺子，下一页的百分数就不会被误读。

13 同一案例中，不受限方法搜索取得更大改善 预算 40 秒

这里选取论文图二十的四种条件中的三种，看前 150 个已评分方法。公共数据 SFT 是 6.0%，自由构造数据 SFT 是 2.9%，不受限研究方法是 23.3%。这是搜索条件的比较，不能把 23.3%全算成刚才蒸馏配方的效果。每个条件只有一次运行，相同方法数量也不代表总成本相同。我们看到的是具体任务中的改善；下一部分换一个环节，看 Agent 怎样协助把评测本身跑清楚。

C 研发流程与训练算法

14 另一项任务：把发布前评测跑清楚 预算 35 秒

OpenAI 工程师 Jeremy Lewi 公开过一项发布前评测工作流。研究员已经有评测目标，但要完成它，还需要找到正确工具、准备命令、处理执行问题，最后解释结果。作者把这些工作放进 Runme 笔记，让 Codex 读取工具说明和之前的记录，再协助推进。这里我们能顺着一次任务往下看：任务怎么提出，谁决定开始执行，结果又怎么留下来。

15 执行之前，研究员先审阅计划 预算 40 秒

屏幕上这句原文很直接：开始之前，等我审阅并批准计划。看下面这条顺序，先有目标和既有记录，Codex 再写出计划，研究员审阅后才执行。为什么这一步有用？因为命令能够跑通，不代表它回答了正确的问题。把计划先写出来，人就能检查准备跑哪些评测、是否漏掉关键条件。这个例子清楚展示了人在执行决策上的位置；它描述的是作者自己的工作流，不能替整个公司作同样的保证。

16 Runme 让下一次任务能接着做 预算 40 秒

Runme 把命令放在笔记里执行，输出也能跟着保存。请从上到下看：目标和计划说明为什么做，命令和输出记录具体发生了什么，解释与取舍交代下一步怎么选。换一个会话，Codex 就能读取这些记录，接着处理任务。它和前面的共享实验记录有一个共同点：上次怎么做、哪里失败、为什么这样决定，都能留给下一次。这里的记忆就保存在这些可读取的记录中。

17 AlphaEvolve 把程序候选交给自动评测 预算 40 秒

Google DeepMind 的 AlphaEvolve 把同样的生成、执行、筛选循环用在程序上。Gemini 先提出候选程序，执行器测正确性和性能，系统保留较好的候选，再继续生成。这里最重要的是，评价可以落到实际运行结果上：程序有没有算对，速度有没有更快。这类反馈适合检查算法改动。它优化的是程序，并不直接重训练模型权重。那么，改一个程序能给大模型研发带来什么收益？下一页看训练里的例子。

18 算子加速 23%，对应总训练时间减少 1% 预算 40 秒

这个案例报告，一个矩阵乘法 kernel 加速了 23%，对应 Gemini 总训练时间减少 1%。左边说的是一个算子，右边说的是完整训练。训练还有其他大量工作，所以一个环节快了很多，整段训练的收益仍可能比较小。23%落在这个算子上，1%才对应总训练时间。这个例子让我们看到，Agent 找到的一项程序改动，怎样实际进入大模型训练。

D 开放研究与递归问题

19 Agent 策略与自身代码，也成为改进对象 预算 40 秒

开放研究还展示了两个改动对象。通义团队的 AgentEvolver 通过生成任务、复用经验和轨迹贡献归因来训练 Agent 策略。Sakana 的 Darwin Gödel Machine 会修改 Agent 自身代码，用基准评价，再保留候选谱系，让不同版本继续探索。策略训练和代码修改发生在不同位置。框架公开以后，这些改动更容易查清；训练数据和模型权重是否开放，还要分别看项目给出的资料。

20 这些案例改变了研发系统的不同部分 预算 40 秒

大家看中间这一列，每个案例到底改了什么。Anthropic 训练的是目标模型，OpenAI 留下的是评测流程和研究记录，Google 修改训练相关程序，开放研究还会改 Agent 策略和代码。再看右边，效果分别落在行为、任务结果或程序性能上。把这些位置分清以后，下一轮研发的问题就具体了：这轮的成果会留在哪里，接下来谁来用它?

21 把上一轮的成果带进下一轮研发 预算 45 秒

先看上面的流程。研究 Agent 做完实验，把方法和结果留下来，下一次尝试就有了起点。前面的共享记录和 Runme，都在做这件事。DGM 还往前走了一步：它改的是 Agent 自身代码，改好的版本可以继续探索新的修改。于是，下一轮既能接着用以前的记录，也可能换成一个改进后的研究 Agent。把问题推进到基础模型，就要继续观察：这样一轮轮做下去，能不能持续研发出更强的后继模型?

22 下一轮，能不能研究得更好？ 预算 35 秒

可以让原版和改进后的版本，各做一轮研究，用同一组新任务，也给同样的时间和算力。先比较它们找到的方法好不好，再换任务、继续迭代几轮。这样能看清几件事：它是不是只擅长刚才那批题，下一轮有没有研究得更有效，还有会不会退步或钻测试的空子。对基础模型研发来说，跟踪这些连续变化，比只看某一轮的最高分更有说服力。

23 AI 已经参与研发，接下来要看下一轮 预算 25 秒

回到开头 Good 提出的问题。今天，Agent 已经能提出训练办法、跑评测、优化程序，也开始改进自身代码。接下来最值得看的，是改进后的研究系统做下一轮研发时，能否继续产出更强的基础模型。沿着这一轮轮的结果往下看，才能理解 AI 辅助研发会走到哪里。

24 谢谢，欢迎提问 预算 5 秒

谢谢大家，欢迎提问。